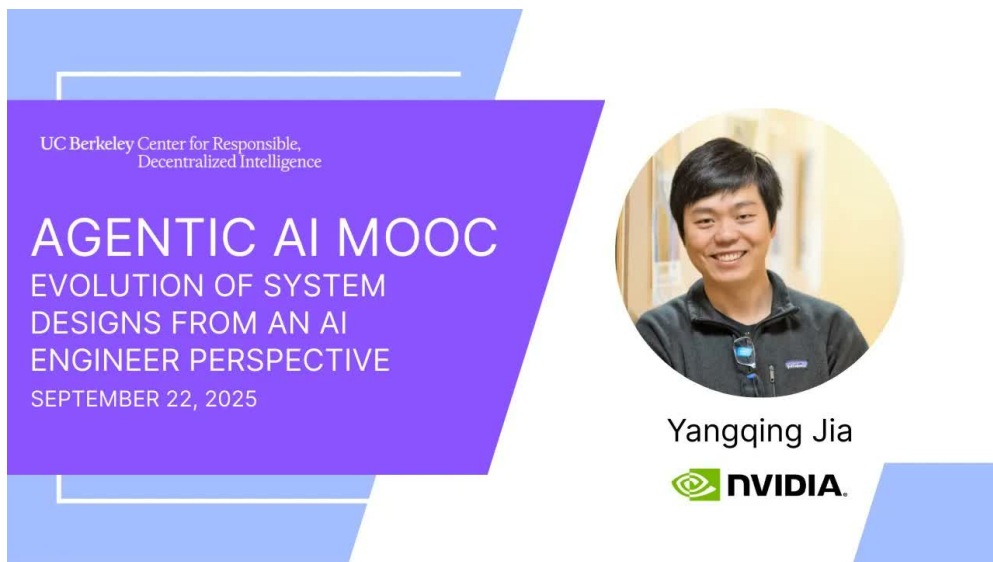


课程笔记

UCB CS294/194-196: Agentic AI (Fall 2025) - Lecture 03: Evolution of System Designs from an AI Engineer Perspective

Yangqing Jia & Codex

2025-09-22



UC Berkeley Center for Responsible,
Decentralized Intelligence

AGENTIC AI MOOC
EVOLUTION OF SYSTEM
DESIGNS FROM AN AI
ENGINEER PERSPECTIVE
SEPTEMBER 22, 2025

Yangqing Jia

 NVIDIA

视频作者/频道: Agentic AI MOOC / UC Berkeley

发布日期: 2025-09-22

视频时长: 1:19:31

视频链接: <https://www.youtube.com/watch?v=xqRAS6rAouo>

目录

1	从研究者到工程师再到创业者	2
1.1	本章小结	2
2	新算法继续推动模型	2
2.1	本章小结	3
3	应用层繁荣: consumer、prosumer 与 enterprise	3
3.1	本章小结	4
4	Fall 2024 的系统前导层: frameworks、compound systems 与 workflow	4
4.1	本章小结	4
5	AI infra 是 IT 的第三根支柱	4
5.1	本章小结	5
6	硬件与软件设计: AI 时代的 back to the future	6
6.1	本章小结	7
7	总结与延伸	7
7.1	本章小结	7

1 从研究者到工程师再到创业者

Yangqing Jia 这一讲的叙事方式非常个人化：他先讲自己在 Berkeley、Google 和创业公司中的经历，再把这些经历抽象成一个工程判断。这个判断就是：**AI 系统的演化方式和传统软件系统不一样**。传统 CS 里，microservices、database、data analytics 的演化路径相对稳定；但 AI 模型、应用和基础设施同时在快速变化，因此 system design 也必须跟着变。

slide 3 用 “Demystify LLM and AGI” 开场，背后的含义并不是否定 AGI，而是提醒大家：很多看似神秘的系统，其实可以用非常朴素的工程原理解释。slide 4 的 “Chinese Typewriter Problem” 是一个极好的类比：中文输入从来不是简单的一键一字，而是通过一个间接编码/选择过程把想法映射到字符。这和今天的 agent / model interface 很像：表面上像 “按键”，底层其实是 representation、encoding、routing 和 state management 的组合。

本讲的风格

这是一场强烈带有产业经验和个人判断的讲座。它的价值在于观察一线系统如何演化，而不是把每句话当成普适定理。阅读时要抓住 “趋势判断” 而不是 “字面语气”。

1.1 本章小结

这一讲不是在讲单一技术点，而是在讲 AI 工程师如何观察产业：模型、应用和基础设施会共同变化，而系统设计必须顺着这个变化方向重构。

2 新算法继续推动模型

slide 8 到 11 的主线很明确：**新算法仍然在推动 LLM models**。讲者列举了 GPT-5、Grok、Gemini、Kimi、DeepSeek、Qwen、GPT-OSS 等例子，并强调 “没有 bubble” 的判断来自消费侧持续增长，而不只是训练侧的 hype。slide 11 把这条线讲得更抽象：GPT、MoE、test-time scaling、reinforcement learning 可以类比到计算机视觉和通用 AI 历史上的结构性创新。

这里最值得记住的，是他把 “LLM” 与 “AGI” 都做了去神秘化。对他来说，今天的 progress 并不来自玄学式的突然顿悟，而是来自多个简单原理的组合：更好的算法、更好的数据、更好的 scale、更好的 evaluation。正因为如此，他才会说历史会重复：AI 的每一轮提升，最终都会回到 “方法论 + 计算” 这个组合上。

这一讲对模型进展的判断

- 新算法仍然会继续推动模型前进；
- 关键变化不只是模型，而是模型与应用、基础设施的协同演化；
- “看起来像 magic” 的系统，往往可以用简单工程原则解释。

值得注意的是，讲者并没有把重点放在 “某个 benchmark 分数涨了多少”，而是放在产业演进曲线上。这种视角对 agent 课程很重要，因为 agent 不是纯研究问题，它天然牵涉模型迭代、系统接口和商业化反馈。

New Algorithms Drive Continued Improvements

My personal opinion and historical analogies...

Date	Nov 2022	Dec 2023	Sep 2024	Jan 2025 (and earlier)
Algorithm	GPT (3.5)	MoE (Mixtral 8x7B)	Test time Scaling	Reinforcement Learning
Analogies	AlexNet (structural innovation)	Ensemble Learning Inception/ResNet	Fully convolutional network Multi-instance learning	General RL GANs

图 1: 新算法持续推动 LLM 的典型轨迹：从 GPT、MoE 到 test-time scaling 与 RL。

2.1 本章小结

模型进步不是单点突破，而是算法、数据和算力三者持续叠加后的结果。理解这一点，才能理解为什么系统设计会被不断重写。

3 应用层繁荣：consumer、prosumer 与 enterprise

slides 13 到 19 转到应用层。这里的核心判断是：**模型和应用的耦合并没有想象中那么强**。slide 14 直接说，perfect app experience correlated, but independent from models。换句话说，模型能力很重要，但产品体验还取决于交互流程、任务分解、记忆、搜索、权限与 workflow。

slide 16 和 17 强调 consumer app landscape is highly fluid, 说明消费级应用会随着 foundational models 的进步快速变化；而 slide 18 和 19 指出 prosumers 的 willingness to pay、enterprise 的慢变需求，才是更可能形成持续价值的区域。讲者把这总结为一个很实在的创业判断：如果你想做垂直领域的 agent，企业 and 专业用户往往比泛消费更容易形成稳定付费。

应用层的三个市场

- **Consumer**：增长快，但 landscape 非常流动；
- **Prosumer**：付费意愿更强，能驱动早期收入；
- **Enterprise**：需求更慢，但更适合把 domain knowledge 与新技术结合成可持续产品。

这部分和 agent 课程的关系非常直接。所谓 agentic workflow，不是把一个聊天框套到每个任务上，而是把 memory、search、tools、human approval 和 business process 重新编排。对于真实产品来说，“模型能力”只是其中一个变量，“应用体验”才是最终收益函数。

3.1 本章小结

应用层的判断是：消费级应用会迅速变化，但 enterprise 与垂直场景更容易形成 agent 的真实价值。模型越强，应用层反而越需要被重新设计。

4 Fall 2024 的系统前导层：frameworks、compound systems 与 workflow

Berkeley Fall 2024 的前导课程对这一讲最有价值的补充，不是再讲一次“AI 会变快”，而是把 agent 系统里最常被口头化的几件事写成了显式工程对象：framework、workflow、compound system、trace 和 state transition。这样一来，**AI engineer perspective** 就不只是产业观察，而是可以落到系统结构上的设计语言。

‘AutoGen’把 multi-agent conversation 变成角色分工、消息路由和终止条件的组合；‘StateFlow’进一步告诉我们，很多 agent 任务需要的是 state-driven execution，而不是无约束对话。‘Building a Multimodal Knowledge Assistant’则把知识接入、图文索引和回答组织看成一条完整 pipeline，而不是单个模型能力。

‘Compound AI Systems & the DSPy Framework’的贡献尤其明显：它把 prompt、demonstration、module composition 和 evaluator 变成可优化对象。对系统设计者来说，这比“写一个更长 prompt”重要得多，因为真正的 agent 方案往往是多个可替换模块的组合，而不是一个不可分解的魔法字符串。

‘Agents for Software Development’与‘AI Agents for Enterprise Workflows’则把执行反馈写进真实作业系统：代码仓库、shell、测试日志、审批状态、企业表单和 trace 都成了 agent 要管理的环境状态。这个视角和本讲的产业判断是相通的：模型能力只是其中一层，真正决定能否落地的是 workflow 设计、状态管理和失败恢复。

Fall 2024 让这讲更像教材的地方

- frameworks 不是聊天壳，而是角色、状态与接口协议；
- compound systems 不是 prompt 堆叠，而是可优化的模块图；
- enterprise workflow 不是 demo，而是带 trace 和 failure recovery 的真实任务系统。

4.1 本章小结

如果只看 Fall 2025，这一讲容易被读成“产业判断”；加上 Fall 2024 的前导层后，它更像一门教材里真正的系统设计章节：模型、应用和基础设施之间，必须通过 framework 与 workflow 连接起来。

5 AI infra 是 IT 的第三根支柱

slide 22 到 32 是这一讲最像系统课的部分。讲者把 AI infra 说成 IT strategy 的 third pillar，并且用历史类比解释它为什么和传统 cloud 不一样。传统路线是 scientific computing、web service cloud、data cloud，而 AI cloud 需要面对的是 exaflops 级算力、异构硬件、高带宽网络和训练/推理共存的资源需求。

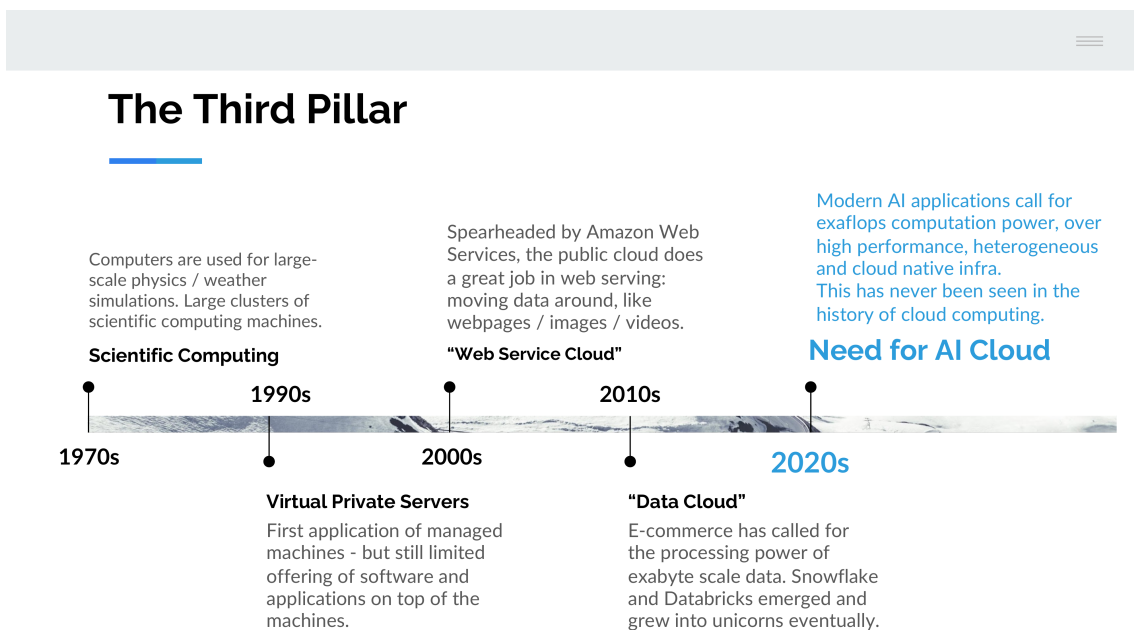


图 2: AI infra 作为 IT 的第三根支柱: 从 scientific computing、web service cloud、data cloud 到 AI cloud 的历史演化。

slide 25 的对比非常有启发性。Data compute 是 $\text{IO} \gg \text{compute}$, Web services 是 $\text{IO} > \text{compute}$, 而 AI compute 是 $\text{compute} \gg \text{IO}$ 。也就是说, AI workload 在算力、分布式结构和工程复杂度上都不同于传统 web workload。它不是更大号的 web service, 而是另一类系统。

slide 27 进一步解释 conventional cloud value proposition 为什么不再完全成立: 传统云强调软件复杂性高、工作负载多样、供应链灵活和 VM 高可互换; 而 AI cloud 里, 软件更像 AI frameworks, workload 更统一, 供应链灵活性更低, 硬件和系统更像为单一类任务深度定制。这也是为什么讲者会说 “AI infra is different, but also the same”: 很多云工程原理仍然有效, 但系统边界已经变了。

AI infra 的新要求

- 面向模型训练与推理的高带宽、高吞吐和低延迟;
- 面向开发者效率的统一平台;
- 面向模型生命周期的 dev / train / inference 一体化;
- 面向供应链和资源调度的多云、弹性与利用率管理。

他对 baremetal 和 K8s 的那句评论不应当机械理解, 而应当读成一个系统判断: 通用云栈并不能自动满足 AI 工作负载的调度、监控、恢复和效率要求。要做 AI infra, 需要更贴近训练、推理和模型生命周期的基础设施抽象。

5.1 本章小结

AI infra 不是把传统云换个名字, 而是围绕 AI workload 重新设计平台。它之所以成为第三根支柱, 是因为 AI 已经有了自己的资源结构、调度结构和商业结构。

6 硬件与软件设计：AI 时代的 back to the future

slide 30 到 32 把讲座收束到系统设计层：what you want to care about 是 developer efficiency 和 infra efficiency。开发者时间宝贵，GPU 也会故障，而且比很多人想的更频繁，所以真正可用的系统必须从开发、训练到推理都能稳定运行。

讲者在这里提出了一个很实用的 best practices 清单：multi-cloud supply chain management、elasticity and utilization management、AI-native platform，以及围绕 model and applications 组建自己的团队。它们都指向同一个事实：AI 系统最难的部分，往往不是算法本身，而是让算法在真实约束里持续工作。

Conventional cloud value proposition no longer holds...

	Conventional Cloud	AI Cloud
Software: Variety	Complicated: Many applications Middleware	Simple: “AI frameworks”
Software: Workload	Varied: Compute, storage, network, big data, database, etc.	Unified: Numerical Computation
Supply Chain: Flexibility	High: CPU based Virtualization/Migration	Low: Large training Hard to live migrate
Supply Chain: Interchangeability	High: VMs can do many different jobs	Low: Really just doing AI compute

图 3: 传统云与 AI 云的差异：软件复杂度、工作负载一致性和供应链属性都发生了变化。

后半段 transcript 还明确提到了更有前瞻性的趋势：一方面，机器人、手机智能和 edge compute 会推动更分布式的 AI 资源部署；另一方面，中心化的大规模训练和推理集群仍然会存在。于是系统设计的真实问题变成了：如何在 centralized scaling 和 edge deployment 之间取得平衡。

讲者的核心工程判断

今天的 AI 系统设计，是在两个方向之间找平衡：

- 一边是更大、更集中、更高效的训练和推理集群；
- 另一边是更靠近设备、更贴近场景、更节能的 edge / robotics / mobile intelligence。

这不是二选一，而是同一产业的两头。

他在最后把这个平衡总结为 exploration 与 exploitation：当大家还在争夺最佳模型时，愿意多烧一些资源；而当工程问题已经清晰时，efficient models、distributed compute 和更贴近部署位置的资源配置会越来越重要。robotics 之所以会再次变得重要，也正是因为它把这条趋势推到了物理世界。

6.1 本章小结

AI 时代的硬件与软件设计，不再只是“更快的服务器”，而是围绕模型、场景和部署位置重新做平衡。中心化算力和 edge 智能会同时增长，这就是讲者说的“back to the future”。

7 总结与延伸

这一讲的真正主题，是从 AI engineer 的视角重新看 system design 的演化。它告诉我们：

1. 新算法仍在推动模型，但模型不再是唯一中心；
2. 应用层繁荣，尤其是 enterprise 和 vertical tasks，决定了 agent 的商业空间；
3. AI infra 已经成为独立的 IT 支柱；
4. 面向 AI 的硬件与软件设计，正在回到“算力、带宽、调度与部署”这些老问题，只是问题规模更大了。

对 agent 课程而言，这一讲提供了一个重要底色：*agents* 不是只靠模型能力就能落地的，它们需要完整的系统生态。也就是说，如果你只看 benchmark 分数，会错过真正决定产品成败的基础设施层。

7.1 本章小结

Lecture 03 的价值在于把产业、应用和基础设施连成一条线。它不是讲某个单点技术，而是在讲 AI 工程师如何判断：哪些问题会被模型改写，哪些问题会被系统重塑。